
Event Extraction from Course Announcements for Calendar Creation

First & Last Name
Student Number, email
[Link to Google Colab / GitHub Repository](#)

Introduction

Course announcements often include schedule changes that are not fixed in the syllabus: quizzes, office hours, midterms, room changes, deadline extensions, and one-time events. Right now, students usually have to read each announcement and copy the useful parts into a calendar by hand. This is slow, and it is easy to miss something important.

This project will build a deep learning system that reads Canvas announcements and turns schedule-related text into structured calendar information for TimeGrid. The model will first classify each announcement as not relevant, notice, deadline, or event. When the announcement contains schedule information, it will also extract fields such as event title, date, time, location, course code, and contact information. A rule-based system can handle simple wording, but announcements are often messy: one message may contain several events, relative dates, or details spread across multiple sentences. A neural model should handle these variations better.

Illustration

Figure 1 shows the planned pipeline. The input announcement is tokenized, passed through a shared Transformer encoder, and then sent to two output heads: one for announcement type and one for token-level field labels.

Announcement text → tokenizer → Transformer encoder → type prediction + token labels
→ structured JSON/ICS output

Figure 1: System overview for announcement classification and event field extraction.

Background & Related Work

The main model is based on the Transformer architecture, which uses self-attention to model relationships between tokens in a sequence (Vaswani et al., 2017). BERT showed that a pretrained bidirectional Transformer can be fine-tuned for many language understanding tasks with only small task-specific heads (Devlin et al., 2019). DistilBERT is a smaller version of BERT that keeps much of the language understanding ability while reducing model size and inference cost (Sanh et al., 2019), which makes it a good candidate for this course project.

Existing event extraction work is also useful, but it does not fully solve this problem. OmniEvent provides a general event extraction toolkit (Li et al., 2023), while this project needs calendar-specific fields such as start time, end time, location, and course context. For that reason, the dataset and labels will be designed around calendar creation rather than general event understanding.

Data Processing

The dataset will be made from Canvas course announcements. Before labeling, all raw text will be anonymized by replacing names, emails, Zoom links, phone numbers, and other sensitive details

with placeholders. The labels will include one announcement type per message and token-level tags for fields such as date, time, title, location, course code, and contact information.

Preprocessing will include duplicate removal, tokenization, sequence padding or truncation, and a train/validation/test split. Since course announcements can be short or long, the maximum sequence length will be chosen after checking the dataset distribution. The final output format will be JSON first, because it is easy to inspect during development, and can later be converted into ICS for calendar integration.

Architecture

The proposed model is a single Transformer-based network implemented in PyTorch. Each announcement is tokenized and encoded by DistilBERT or a small BERT model. The shared encoder output is then used by two heads. The first head performs announcement-type classification using the pooled representation. The second head performs token labeling, assigning a field tag to each token.

This design keeps the system as one model while supporting both tasks. It also lets the model share useful language features between classification and extraction. During inference, the predicted token labels will be grouped back into fields, normalized where possible, and returned as structured event data.

Baseline Model

The baseline will be a rule-based system using chrono-node, regular expressions, and keyword matching. Chrono-node is an open-source natural language date parser for JavaScript (chrono-node, 2026). It will extract and normalize dates and times, while regular expressions and keyword rules will identify event phrases and simple fields such as title, start time, and end time. Duckling will also be considered for date and time parsing because it extracts structured values from natural language text (Duckling, 2026).

The baseline and neural model will be compared using announcement-type accuracy and macro F1-score. For field extraction, I will use token-level precision, recall, and F1-score. This comparison should show whether the Transformer model improves over a simple rules-first approach.

Ethical Considerations

This project uses course announcement data, so privacy is a real concern. Raw announcements will not be uploaded to a public repository, and all training examples will be anonymized before use. The system can also make mistakes, especially with relative dates or ambiguous times. For that reason, it should be treated as an assistive tool: it can suggest calendar events, but the user should still confirm them before saving.

References

References

- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017. <https://arxiv.org/abs/1706.03762>.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*, 2019. <https://arxiv.org/abs/1810.04805>.
- V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*, 2019. <https://arxiv.org/abs/1910.01108>.
- S. Li et al. OmniEvent: A comprehensive, fair, and easy-to-use toolkit for event understanding. *arXiv:2309.14258*, 2023. <https://arxiv.org/abs/2309.14258>.
- wanasit. chrono-node: A natural language date parser in JavaScript. GitHub repository. <https://github.com/wanasit/chrono>.
- Meta/Facebook. Duckling: Language, engine, and tooling for expressing, testing, and evaluating composable language rules on input strings. GitHub repository. <https://github.com/facebook/duckling>.